Solution: Toeplitz Matrix

Problem Statement

Given an `m x n` matrix, determine if the given matrix is a `Toeplitz matrix`.

A Toeplitz matrix is one in which every diagonal from `top-left` to `bottom-right` has the same elements.

In other words, the matrix should have the property that each element is equal to the element diagonally down to its right.

Example 1:

- **Input**:

```
[[1,2,3],
 [4,1,2],
 [5,4,1]]
```

- **Expected Output**: true
- **Justification**: All diagonals have the same elements. Diagonals of the above matrix are [5], [4, 4], [1, 1, 1], [2, 2], and [3].

Example 2:

- **Input**:

```
[[1,2],
 [3,4]]
```

- **Expected Output**: false
- **Justification**: The diagonal are [3], [1, 4], and [2]. A diagonal [1, 2] doesn't have the same elements.

Example 3:

- **Input**:

```
[[7,7,7],
 [7,7,7],
 [7,7,7]]
```

- **Expected Output**: true
- **Justification**: Each diagonal, although containing the same repeated element (7), satisfies the condition of a Toeplitz matrix.

Solution

To solve this problem, we will iterate through the elements of the matrix, checking if each element (except those in the last row and last column) matches with its diagonal successor. This approach works because a Toeplitz matrix is defined by the consistency of elements along its diagonals. By verifying this condition, we can ensure whether the matrix adheres to the Toeplitz criteria.

Step-by-step Algorithm

- Iterate over each row of the matrix, except the last row.
  - For each row, iterate over each column, except the last column.
    - Check if the current element is equal to the element diagonally down to its right.
    - If any such pair of elements is not equal, return false immediately.
- If the entire matrix is iterated without finding any mismatch, return true.

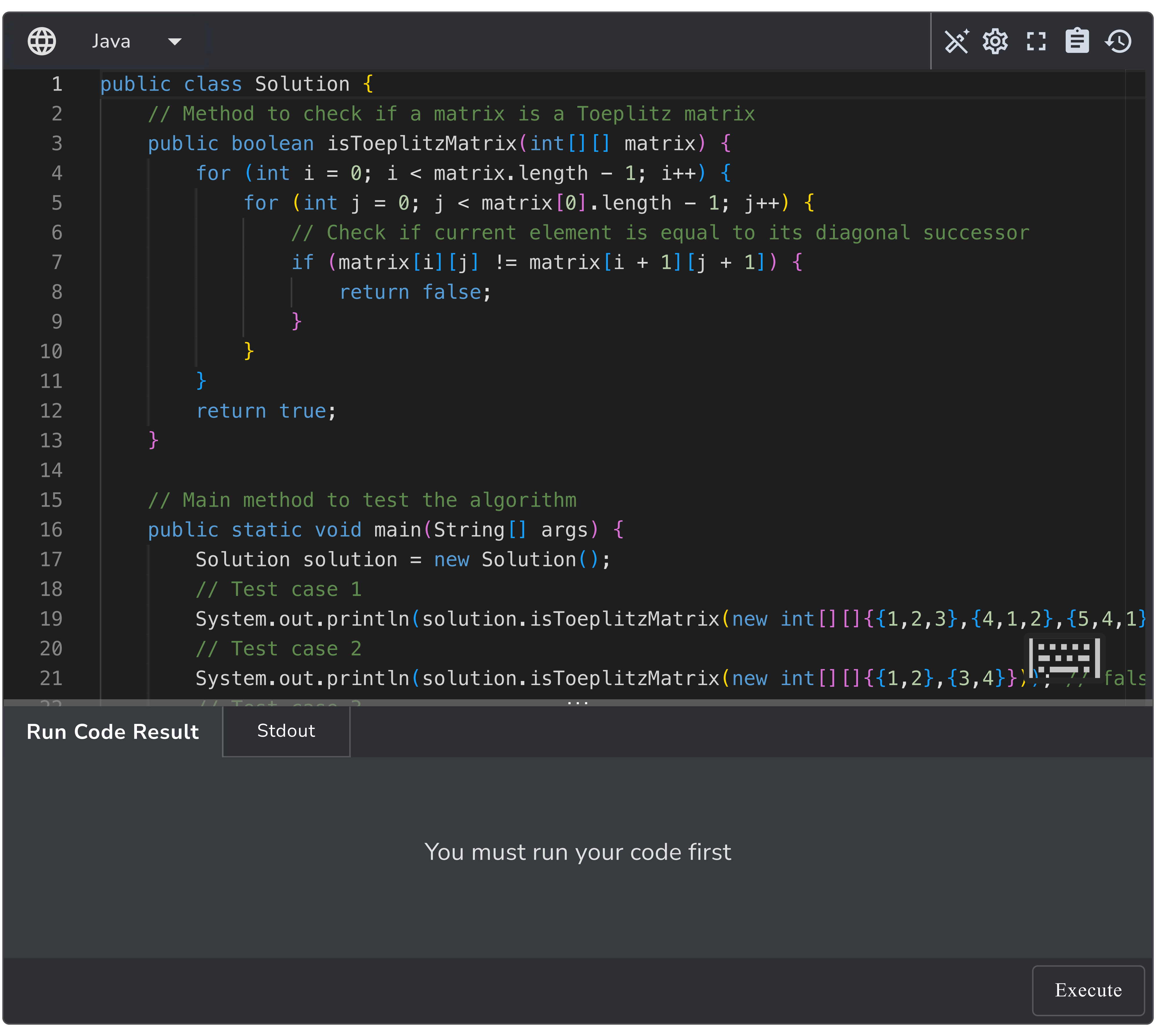
Algorithm Walkthrough

Using Example 1: matrix =

```
[[1,2,3],
 [4,1,2],
 [5,4,1]]
```

- Start with the first element `(1)` at matrix[0][0]. It matches with its diagonal successor `(1)` at matrix[1][1]. Continue.
- Move to the second element `(2)` in the first row at matrix[0][1]. It matches with its diagonal successor `(2)` at matrix[1][2]. Continue.
- Skip the last element `(3)` in the first row as it has no diagonal successor.
- Move to the first element `(4)` in the second row. It matches with its diagonal successor `(4)`. Continue.
- Next, Move to the second element `(1)` in the second row. It matches with its diagonal successor `(1)`. Continue.
- Skip the last element `(2)` in the second row.
- Since we've reached the last row, we conclude that the matrix is a Toeplitz matrix.

Code



```java
public class Solution {
    // Method to check if a matrix is a Toeplitz matrix
    public boolean isToeplitzMatrix(int[][] matrix) {
        for (int i = 0; i < matrix.length - 1; i++) {
            for (int j = 0; j < matrix[0].length - 1; j++) {
                // Check if current element is equal to its diagonal successor
                if (matrix[i][j] != matrix[i + 1][j + 1]) {
                    return false;
                }
            }
        }
        return true;
    }

    // Main method to test the algorithm
    public static void main(String[] args) {
        Solution solution = new Solution();
        // Test case 1
        System.out.println(solution.isToeplitzMatrix(new int[][]{{1,2,3},{4,1,2},{5,4,1}
        // Test case 2
        System.out.println(solution.isToeplitzMatrix(new int[][]{{1,2},{3,4}})); // fals
```

You must run your code first

Complexity Analysis

- **Time Complexity**: $O(m * n)$, where `m` is the number of rows and `n` is the number of columns in the matrix. Each element is checked once.
- **Space Complexity**: $O(1)$, as no extra space is used besides a few variables for iteration.